Alamein International University

Faculty of Computer Science & Engineering

CSE373 – Parallel Programming

Parallel Web Crawler

Project Reference

All Phases · What We Built · How Parallel Programming Is Applied

Team: [Ragnarok] | Course: CSE373 | 2025-2026

1. What We Built

We built a **Parallel Web Crawler** with a live JavaFX GUI. A web crawler automatically visits web pages, collects information from them, and follows links to visit more pages. Our crawler does this using **multiple threads running at the same time**, which is the core parallel programming concept.

Instead of visiting pages one by one (sequential), our system sends **N HTTP requests simultaneously** using a thread pool. This is similar to how you can download multiple files at the same time instead of waiting for each one to finish before starting the next.

1.1 Project Idea vs Course Suggestion

The course suggested a 'Parallel Image Downloading and Processing Pipeline'. Our team chose an **original alternative idea**: a Parallel Web Crawler. This is allowed under the course policy — alternative ideas need to satisfy the same core parallel requirements.

Requirement	Suggested Idea	Our Project
Parallel downloading	Download ZIP files in parallel	Fetch web pages in parallel with thread pool
Multi-stage pipeline	Download → Extract → Process images	Seed → Discover → Fetch → Extract → Store
Synchronization	File I/O synchronization	ConcurrentHashMap + AtomicInteger + synchronized
Performance comparison	Sequential vs parallel time	Sequential mode (1 thread) vs parallel (N threads)
Error handling	Retry failed downloads	Retry logic: failed pages retried once
Bonus: Dashboard	Progress dashboard	Full JavaFX live GUI: stats, table, log, progress bar

1.2 The 5-Stage Processing Pipeline

Our crawler processes every website through exactly 5 stages. Each stage feeds into the next — this is what makes it a **pipeline**:

#	Stage	What Happens	Where in Code
1	**Seed Input**	User enters starting URL → added to BlockingQueue as depth=0 task	CrawlerEngine.start()
2	**URL Discovery**	BFS: links extracted from each page, new ones added to queue if not visited	crawlPageWithRetry() queue.offer()
3	**HTTP Fetch**	N threads simultaneously send HTTP GET requests — this is the parallel stage	runParallel() executor.submit()

#	Stage	What Happens	Where in Code
4	**Data Extraction**	Each thread parses HTML: title, meta description, all outbound links, timing	crawlPageWithRetry() doc.title() / select()
5	**Storage**	Results written concurrently to CSV + JSON + SQLite	saveAndNotify() CSV/JSON/SQLite Storage

2. Submission Phases

Phase 1 — Requirements & Analysis

Goal: Define the project, prove it satisfies parallel requirements, submit UML diagrams.

- Project requirements document: what the system must do (12 functional requirements)
- System analysis: architecture decisions, parallel concepts selected
- 6 UML diagrams: Use Case, Class, Sequence, Activity, Component, State
- Team division: who owns which files and responsibilities

Status: ✅ **Complete**

Phase 2 — Parallel Crawling + Extraction

Goal: Implement the actual parallel crawling, extraction, and retry logic. Deliver working code + report.

- **Parallel fetching:** FixedThreadPool sends N HTTP requests simultaneously
- **Parallel extraction:** Each thread independently parses its own HTML document
- **Retry logic:** crawlPageWithRetry() retries once on IOException before marking as error
- **Sequential mode:** threadCount=1 triggers runSequential() for benchmark
- **BenchmarkResult record:** stores per-stage timing for Phase 3 report

Status: ✅ **Complete**

Phase 3 — Full System + Performance Report

Goal: Add benchmark mode, stage timing, speedup calculation. Write final report with performance comparison.

- **Benchmark Mode:** GUI runs sequential then parallel, shows Speedup = T_seq / T_par
- **Stage timing:** Discovery, Fetch, Extraction, Storage each timed separately in ms
- **Speedup display:** Efficiency = Speedup / threadCount × 100% shown in GUI
- **Final report:** Performance table + chart + discussion of bottlenecks
- **Output samples:** Sample CSV, JSON, SQLite output files for submission

Status: ⬜ **In Progress**

Evaluation Criteria vs Our Implementation

Criterion	Points	How We Satisfy It
Parallel Downloading	**20**	ExecutorService.submit() runs N concurrent HTTP fetches — same as downloading N files at once
Parallel Extraction	**20**	Each thread extracts data (title, links, desc) from its own HTML document simultaneously
Parallel Processing	**30**	Multi-stage pipeline: concurrent storage to 3 formats, atomic dedup, synchronized writers

Criterion	Points	How We Satisfy It
Error Handling	**10**	Retry logic (crawlPageWithRetry), IOException/TimeoutException handling, graceful stop
Code Quality & Documentation	**10**	JavaDoc comments, clean package structure, Builder pattern, meaningful variable names
Project Report	**10**	This document + code explanation file cover all required report sections
BONUS: Progress Dashboard	**+10**	JavaFX live GUI: real-time stats, log, results table, progress bar, stage timing — fully built

3. How Parallel Programming Is Applied

This section explains every parallel programming concept used in the project — what it is, why we chose it, and exactly where in the code it appears.

3.1 Thread Pool — ExecutorService

What it is: A pool of N worker threads that are created once and reused for many tasks. Instead of creating a new thread for every URL (expensive: memory + OS scheduling overhead), we create N threads and give them work from a queue.

Why we use it: Web crawling is I/O-bound — most time is spent waiting for HTTP responses (100ms–2000ms per page). With N threads waiting on different servers simultaneously, total time drops from T_single × total_pages to roughly T_single × (total_pages / N).

Where in code: CrawlerEngine.runParallel() — lines creating ExecutorService and submitting tasks

```
// Create N reusable worker threads (N = config.getThreadCount(), default 6)
ExecutorService executor = Executors.newFixedThreadPool(config.getThreadCount());

// Submit a crawl task — returns immediately with a Future (promise of a result)
Future<CrawledPage> future = executor.submit(() -> crawlPageWithRetry(task, 0));

// Wait for the result — blocks until done or timeout
CrawledPage page = future.get(config.getTimeoutMs() + 3000, TimeUnit.MILLISECONDS);
```

3.2 BlockingQueue — Producer-Consumer Pattern

What it is: A thread-safe queue where: the producer (main loop) adds URL tasks, and the consumers (worker threads) take tasks and process them. 'Blocking' means threads wait automatically when the queue is empty — no busy-waiting or manual sleep loops needed.

Why we use it: It decouples URL discovery from URL processing. While some threads are fetching pages, others can discover and enqueue new URLs without locking or coordination.

```
// PRODUCER: main loop adds discovered URLs to the queue
queue.offer(new UrlTask(href, task.depth() + 1));

// CONSUMER: main loop drains a batch from the queue for the thread pool
List<UrlTask> batch = new ArrayList<>();
queue.drainTo(batch, config.getThreadCount() * 2);
```

3.3 ConcurrentHashMap — Lock-Free Thread-Safe Set

What it is: A hash map that allows multiple threads to read and write simultaneously without blocking each other. We use it as a set of visited URLs.

The key insight: With a regular HashSet, two threads could both check 'is this URL visited?' at the same time, both see 'no', and both crawl the same URL. ConcurrentHashMap's add() is atomic — it checks AND inserts in one indivisible step.

```
// This set allows safe concurrent access from all N threads
Set<String> visitedUrls = ConcurrentHashMap.newKeySet();

// ATOMIC: check-and-insert in one step
// Returns true only if the URL was NOT already in the set
if (visitedUrls.add(href)) {
    // Only ONE thread will reach here for each unique URL
    queue.offer(new UrlTask(href, task.depth() + 1));
}
```

3.4 AtomicInteger + AtomicBoolean — Lock-Free Counters

What it is: Variables that can be safely read and modified by multiple threads simultaneously using CPU-level Compare-And-Swap (CAS) — no locks required.

Without AtomicInteger: Two threads do count++ at the same time. Both read value=5, both compute 5+1=6, both write 6. We lost one increment. With AtomicInteger, incrementAndGet() is one atomic CPU instruction.

```
AtomicInteger pageCount = new AtomicInteger(0);
AtomicBoolean stopped   = new AtomicBoolean(false);

// Safe from any thread — never loses an increment
pageCount.incrementAndGet();

// Stop signal readable by all threads instantly
stopped.set(true);  // in onStop()
if (stopped.get()) return null;  // in each worker thread
```

3.5 Synchronized Methods — Storage Writers

What it is: The synchronized keyword ensures only one thread at a time executes that method. All other threads wait. Used in our storage classes to prevent garbled output.

Why needed: If two threads write to the CSV file at the same time without synchronization, their output could interleave — one row could be written in the middle of another row.

```
// In CsvStorage.java — only one thread writes at a time
public synchronized void save(CrawledPage page) {
    writer.printf("\"%s\",\"%s\",%d,%d%n",
        page.getUrl(), page.getTitle(),
        page.getStatusCode(), page.getCrawlTimeMs());
    writer.flush();
}
```

3.6 Platform.runLater() — JavaFX Thread Safety

What it is: JavaFX requires all UI updates (labels, tables, text areas) to happen on the Application Thread. Our crawler runs on background threads. Platform.runLater() schedules UI updates to run on the correct thread.

```
// Called from worker thread — cannot touch UI directly here
engine.setOnPageCrawled(page -> {
    // Schedule this block to run on the JavaFX Application Thread
    Platform.runLater(() -> {
        tableData.add(new PageRow(page));          // ✅ Safe
        statPages.setText(String.valueOf(count));  // ✅ Safe
        logArea.appendText("[✓] " + page.getUrl());// ✅ Safe
    });
});
```

3.7 Retry Logic — Phase 2 Enhancement

What it is: When an HTTP request fails with an IOException (network error, server down, timeout), instead of immediately marking the page as failed, we try again once. This handles temporary network issues.

```
private CrawledPage crawlPageWithRetry(UrlTask task, int attempt) {
    try {
        // ... HTTP request ...
        return new CrawledPage(url, title, desc, links, 200, time, depth);
```

```
    } catch (IOException e) {
        if (attempt < MAX_RETRIES) {  // MAX_RETRIES = 1
            // Retry once: call self with attempt+1
            return crawlPageWithRetry(task, attempt + 1);
        }
        // All retries failed: return error page
        return new CrawledPage(url, "ERROR", e.getMessage(),
            emptyList(), 0, 0, depth);
    }
}
```

3.8 Sequential vs Parallel Benchmark — Phase 2+3

What it is: To prove our parallel implementation is faster, we run the same crawl twice: once single-threaded (threadCount=1) and once with N threads. We measure wall-clock time for each and calculate speedup.

```
// Speedup = how many times faster parallel is vs sequential
double speedup = (double) seqTotalMs / parTotalMs;

// Efficiency = how well we use each thread (100% = perfect linear scaling)
double efficiency = (speedup / threadCount) * 100;

// Example output:
// Sequential: 60,000 ms
// Parallel (6 threads): 12,500 ms
// Speedup: 4.8x   Efficiency: 80%
```

4. Project Files Overview

File	Package	Role
Main.java	com.crawler	Entry point — creates CrawlerConfig and starts the engine (headless/no GUI mode)
CrawlerEngine.java ★	core	THE core class — thread pool, BFS queue, retry logic, stage timing, all parallel logic
CrawlerConfig.java	model	All crawl settings — Builder pattern (seedUrl, threads, depth, timeout...)
CrawledPage.java	model	Data model for one crawled page (url, title, links, statusCode, timing, depth)
CsvStorage.java	storage	Writes one CSV row per page — synchronized save() method
JsonStorage.java	storage	Writes NDJSON per page + summary JSON at the end — synchronized
SqliteStorage.java	storage	Writes to pages + links tables in SQLite — synchronized + batch insert
CrawlerApp.java	gui	JavaFX Application class — loads FXML, applies CSS, opens window
MainController.java ★	gui	GUI event handler — START/STOP/CLEAR, real-time updates, benchmark mode, speedup display
ConsoleDisplay.java	util	Colored terminal output for headless mode (printBanner, printSummary)

★ = files with Phase 2+3 enhancements (retry logic, sequential mode, benchmark)